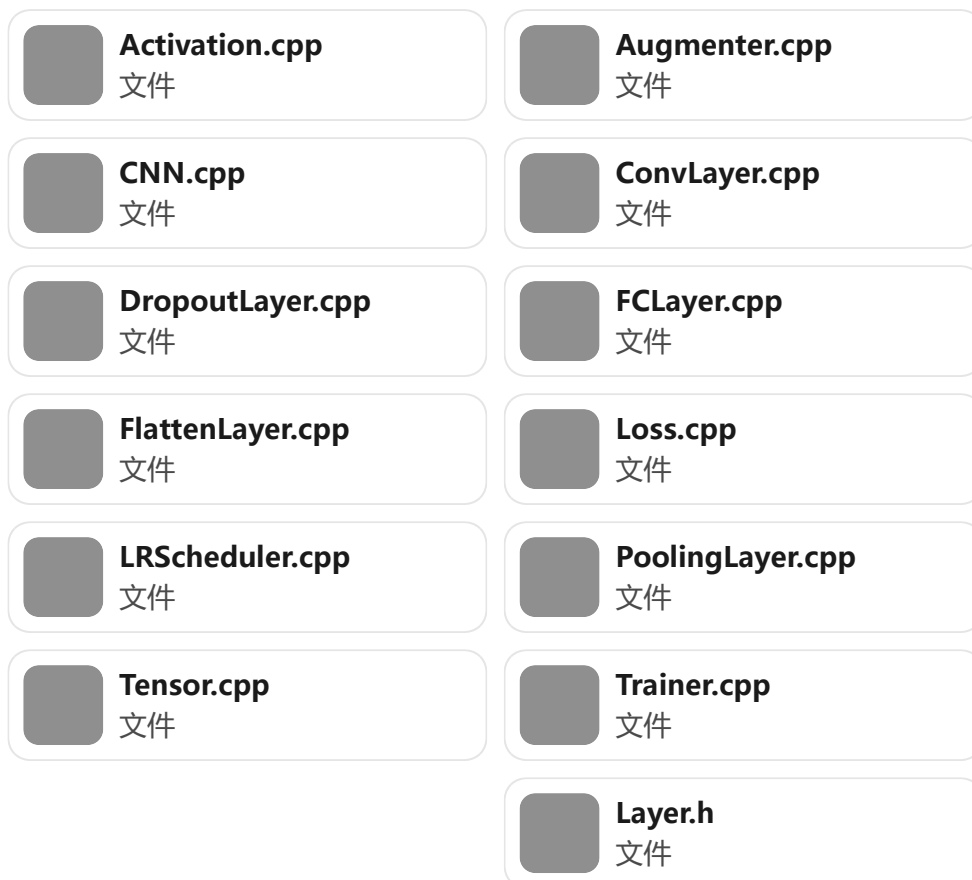




这是我当前的项目中的CPP文件，都有对应的头文件（此处省略），只有一个layer是只有头文件没有CPP，我不知道为什么。

我现在需要你做为我的CPP导师，来给我讲解这些代码，以便我的汇报。

我先把这些 `.cpp` 和唯一上传的 `Layer.h` 按“模块职责—数据流—汇报讲法”梳理一遍。重点会放在你能对老师讲清楚：每个文件干什么、CNN训练/推理怎么走、为什么有的类只有头文件没有 `cpp`。

我已经看完核心结构了。这个项目的特点很明显：它没有用 PyTorch 这类框架，而是自己实现了 Tensor、Layer、多层 CNN、反向传播、SGD/Momentum、学习率调度、数据增强、训练日志和模型保存。

已思考 1m 2s

可以。你这套代码**适合按“自研简易 CNN 框架”来汇报**，不要逐行念代码，而是讲清楚：它如何表示数据、如何把层串起来、如何前向传播、如何反向传播、如何训练、如何保存模型。

1. 先回答你最疑惑的：为什么 `Layer` 只有头文件，没有 CPP?

因为 `Layer.h` 里面的 `Layer` 是一个抽象基类 / 接口类。

它主要规定所有层都必须实现：

C++

```
virtual Tensor forward(const Tensor& input) = 0;
virtual Tensor backward(const Tensor& outputGradient) = 0;
virtual std::string type() const = 0;
```

这里的 `= 0` 表示**纯虚函数**，意思是：

`Layer` 自己不真正工作，它只是规定“所有神经网络层都必须有 forward、backward、type 这些接口”。

同时，里面有一些默认函数，比如：

C++

```
virtual void zeroGrad() {}
virtual void train(bool enabled) { static_cast<void>(enabled); }
virtual bool isTrainable() const noexcept { return false; }
```

这些函数已经直接在类里面写完了。

在 C++ 中，写在类定义内部的函数默认就是 inline 实现，所以不需要再单独放到

`Layer.cpp`。

所以你汇报时可以说：

`Layer` 是整个网络层系统的统一接口，它本身不代表具体层，因此不需要单独的 cpp 文件。具体实现分别在卷积层、池化层、全连接层、激活层等 cpp 文件中完成。

2. 整个项目的代码框架

你可以把这些文件分成五类：

类别	文件	作用
数据结构基础	<code>Tensor.cpp</code>	表示三维张量，也就是图片、特征图、向量

类别	文件	作用
神经网络层	ConvLayer.cpp, FCLayer.cpp, PoolingLayer.cpp, FlattenLayer.cpp, Activation.cpp, DropoutLayer.cpp	实现 CNN 的各类层
网络组装	CNN.cpp	把多个 Layer 串成完整 CNN
训练系统	Trainer.cpp, Loss.cpp, LRScheduler.cpp	负责训练、损失、准确率、学习率调整、验证、保存 checkpoint
数据增强	Augmenter.cpp	训练时对图像做旋转、平移、缩放、亮度、对比度、噪声增强

整体数据流是：

```

图片
↓
DataLoader 读取成 Tensor(3, 32, 32)
↓
CNN.forward()
↓
Conv → ReLU → Pool → Conv → ReLU → Pool → Flatten → FC → ReLU → FC → Softmax
↓
输出每个类别的概率
↓
CrossEntropyLoss 计算损失
↓
CNN.backward()
↓
反向传播梯度
↓
update / updateWithMomentum 更新参数

```

3. `Tensor.cpp`：整个框架的数据容器

`Tensor` 是这个项目里最基础的数据结构。

它保存三个维度：

```

C++

channels_
height_
width_

```

以及真正的数据：

```
C++  
  
std::vector<float> values_;
```

也就是说，一张 RGB 图片会被表示成：

```
3 × 32 × 32
```

其中：

- `channels = 3`：RGB 三通道；
- `height = 32`；
- `width = 32`；
- `values_` 里面连续存放所有像素值。

访问某个元素时，用：

```
C++  
  
at(channel, row, column)
```

它内部把三维坐标转成一维下标：

```
C++  
  
(channel * height + row) * width + column
```

汇报时可以这样说：

`Tensor` 是项目中统一的数据表示方式。无论是输入图像、中间特征图，还是全连接层输出的分类向量，本质上都用 `Tensor` 保存。它内部用 `vector<float>` 连续存储数据，这样实现简单，也方便进行前向传播和反向传播。

4. `Layer.h`：所有层的统一接口

`Layer` 的作用是让 CNN 可以用统一方式管理不同层。

比如在 `CNN.cpp` 里，所有层都存在：

```
C++  
  
std::vector<std::unique_ptr<Layer>> layers_;
```

虽然里面实际可能是：

```
ConvLayer  
ReLULayer  
MaxPoolingLayer  
FlattenLayer  
FCLayer  
SoftmaxLayer  
DropoutLayer
```

但是它们都继承自 `Layer`，所以 CNN 可以统一调用：

```
C++  
  
output = layer->forward(output);
```

反向传播时统一调用：

```
C++  
  
gradient = layer->backward(gradient);
```

这就是 C++ 的**多态**。

汇报时可以说：

这个项目用抽象基类 `Layer` 统一了不同神经网络层的接口。CNN 不需要关心当前层到底是卷积层、池化层还是全连接层，只要调用统一的 `forward` 和 `backward` 即可。这体现了面向对象设计中的多态思想。

5. `ConvLayer.cpp`：卷积层

这是 CNN 里最核心的层。

构造函数里初始化：

```
C++  
  
weights_  
biases_  
weightGradients_  
biasGradients_  
weightVelocity_  
biasVelocity_
```

其中：

- `weights_`：卷积核参数；
- `biases_`：偏置；
- `weightGradients_`：权重梯度；
- `biasGradients_`：偏置梯度；
- `weightVelocity_`, `biasVelocity_`：Momentum 优化器用的速度项。

初始化方式是 He 初始化：

```
C++  
  
sqrt(2.0F / (inputChannels * kernelSize * kernelSize))
```

这个初始化方式比较适合 ReLU 网络。

卷积层 forward 做什么？

核心逻辑是：

```
对每个输出通道  
    对每个输出位置  
        sum = bias  
        遍历输入通道  
            遍历卷积核窗口  
                sum += 输入像素 × 卷积核权重
```

它支持：

- 多输入通道；
- 多输出通道；
- kernel size；
- stride；
- padding。

输出尺寸公式是：

```
outputHeight = (inputHeight + 2 * padding - kernelSize) / stride + 1  
outputWidth  = (inputWidth  + 2 * padding - kernelSize) / stride + 1
```

汇报时可以说：

卷积层通过多个可训练卷积核在图像或特征图上滑动，提取局部特征。每个输出通道对应一组卷积核，最终得到新的特征图。

卷积层 backward 做什么？

反向传播时它做三件事：

第一，计算 bias 的梯度：

```
C++  
biasGradients_[outputChannel] += gradient;
```

第二，计算 weight 的梯度：

```
C++  
weightGradients_[weightIndex] += inputValue * gradient;
```

第三，把梯度继续传回输入：

```
C++  
inputGradient += weight * gradient;
```

也就是说，卷积层既要更新自己的参数，也要把梯度传给前一层。

汇报时可以说：

反向传播时，卷积层根据输出梯度同时计算三类信息：卷积核权重的梯度、偏置的梯度，以及传回上一层的输入梯度。这样训练过程就能通过梯度下降更新卷积核，使其逐渐学会提取有效特征。

6. `FCLayer.cpp`：全连接层

全连接层可以理解成普通神经网络里的线性层：

```
output = W × input + b
```

它的输入一般来自 `FlattenLayer` 展平后的向量。

forward 中：

```
C++  
sum += weights_[rowOffset + inputIndex] * input[inputIndex];
```

backward 中：

C++

```
weightGradients += input * gradient  
biasGradients += gradient  
inputGradient += weight * gradient
```

它和卷积层一样，也支持：

- 普通 SGD 更新；
- Momentum 更新；
- 权重衰减 `weightDecay`；
- 保存优化器状态。

汇报时可以说：

全连接层负责把前面卷积层提取到的特征映射到具体类别上。卷积层更像是在提取图像特征，而全连接层更像是在根据这些特征做分类判断。

7. `Activation.cpp`：ReLU 和 Softmax

这个文件实现了两个激活层。

ReLU

ReLU 的公式是：

```
max(0, x)
```

forward 时：

C++

```
output[index] = std::max(0.0F, input[index]);
```

同时它保存了一个 `active_` 掩码：

C++

```
active_[index] = input[index] > 0 ? 1 : 0;
```

backward 时：

C++

```
inputGradient[index] = active ? outputGradient[index] : 0;
```

也就是说：

- 如果 forward 时这个神经元大于 0，梯度正常传回；
- 如果小于等于 0，梯度变成 0。

汇报时可以说：

ReLU 给网络引入非线性，使 CNN 不只是线性变换的堆叠。同时它在反向传播中只允许正激活区域传递梯度。

Softmax

Softmax 用在最后一层，把输出转成概率分布。

forward 中先找最大值：

```
C++  
  
maximum = max(input)
```

然后计算：

```
C++  
  
exp(input[index] - maximum)
```

这里减去最大值是为了防止 `exp` 数值溢出。

最后除以总和：

```
C++  
  
probability /= sum
```

输出结果就是每个类别的概率。

backward 里实现了 Softmax 的梯度传播。

汇报时可以说：

Softmax 把模型最后输出的分数转成概率，使每一类的概率加起来为 1。代码中通过减去最大值避免指数运算溢出，这是常见的数值稳定技巧。

8. PoolingLayer.cpp：最大池化层

这个项目实现的是 `MaxPoolingLayer`。

forward 时，在每个窗口中取最大值：

2×2 区域 → 取最大值

同时保存最大值的位置：

C++

```
maximumIndices_
```

backward 时，梯度只传回当初最大值所在的位置：

C++

```
inputGradient[maximumIndices_[index]] += outputGradient[index];
```

汇报时可以说：

最大池化层用于降低特征图尺寸，减少计算量，同时保留局部区域中最显著的特征。反向传播时，只有 forward 阶段被选中的最大值位置会接收梯度。

9. FlattenLayer.cpp：展平层

这个层很简单，但很关键。

forward：

C++

```
Tensor output(1, 1, input.size());  
output.values() = input.values();
```

它把三维特征图：

channels × height × width

变成一维向量：

1 × 1 × size

backward 时再恢复原来的三维形状。

汇报时可以说：

Flatten 层本身不做学习，只负责形状转换。它把卷积层输出的三维特征图拉平成一维向量，以便送入后面的全连接层。

10. DropoutLayer.cpp：防止过拟合

Dropout 只在训练时启用。

forward 时，它会随机丢弃一部分神经元：

```
C++  
  
std::bernoulli_distribution distribution(1.0F - rate_);
```

保留下来的值会乘以：

```
C++  
  
scale_ = 1.0F / (1.0F - rate_)
```

这叫 **inverted dropout**。

它的好处是：

- 训练时随机丢弃部分特征，减少模型对某些神经元的依赖；
- 推理时不需要额外缩放，直接原样通过。

推理时：

```
C++  
  
if (!training_) {  
    copy input to output;  
}
```

汇报时可以说：

Dropout 是一种正则化手段。训练时随机屏蔽部分神经元，迫使模型学习更鲁棒的特征；推理时关闭 Dropout，保证预测稳定。

11. `Loss.cpp`: 损失函数和评估指标

这个文件包括:

```
CrossEntropyLoss  
Accuracy  
PerClassAccuracy  
ConfusionMatrix  
argmax
```

CrossEntropyLoss

损失函数是交叉熵:

```
C++  
  
-log(probabilities[label])
```

也就是:

正确类别的预测概率越大, 损失越小; 正确类别概率越小, 损失越大。

代码里用了:

```
C++  
  
std::max(probabilities[label], 1.0e-7F)
```

这是为了避免 `log(0)`。

Accuracy

`Accuracy` 用来统计整体准确率。

核心逻辑:

```
C++  
  
if (argmax(probabilities) == label) {  
    ++correct_;  
}  
  
++total_;
```

PerClassAccuracy

它统计每个类别自己的准确率。

这对交通标志识别很有用，因为某些类别样本可能少，整体准确率高不代表每一类都识别得好。

ConfusionMatrix

混淆矩阵记录：

```
预测成某类 / 实际是某类
```

它可以看出模型经常把哪些类别混淆。

汇报时可以说：

除了整体准确率，代码还实现了按类别准确率和混淆矩阵，这可以更细致地分析模型在不同交通标志类别上的表现。

12. LRScheduler.cpp：学习率调度器

学习率不是一直不变的，而是可以随 epoch 变化。

代码支持几种策略：

```
Step  
MultiStep  
Cosine  
Warmup
```

Warmup 的意思是：

训练刚开始时，学习率从较小值逐渐升到初始学习率，避免一开始更新过猛。

Step / MultiStep 是周期性衰减：

```
C++  
  
lr *= decayFactor ^ k
```

Cosine 是余弦退火：

C++

```
lr = minLR + (initialLR - minLR) * cosine
```

汇报时可以说：

学习率调度器用于控制训练过程中参数更新的步长。前期可以用较大学习率快速收敛，后期逐渐减小学习率，使模型更稳定地接近较优解。

13. Augmenter.cpp：数据增强

这个文件负责对训练图片做随机变换。

它支持：

旋转
平移
缩放
亮度变化
对比度变化
噪声

其中旋转和缩放用了双线性插值：

C++

```
sampleChannel(...)
```

双线性插值的作用是：

当旋转或缩放后，目标像素对应到原图的位置可能不是整数坐标，所以需要根据周围四个像素加权计算。

最后把所有像素限制在 `[0, 1]`：

C++

```
val = std::max(0.0F, std::min(1.0F, val));
```

汇报时可以说：

数据增强可以人为扩充训练样本的多样性，使模型适应交通标志在真实场景中的角度、位置、亮度、噪声变化，从而提高泛化能力。

14. CNN.cpp：网络主体

这个文件负责把所有层串起来。

构造函数中支持两种结构：

第一种：LeNet 风格

输入是：

```
3 × 32 × 32
```

结构大致是：

```
Conv(3→6, 5×5)
ReLU
MaxPool
Conv(6→16, 5×5)
ReLU
MaxPool
Flatten
FC(400→120)
ReLU
FC(120→classCount)
Softmax
```

它是经典轻量 CNN，参数少，适合教学展示。

第二种：Enhanced 结构

结构更深：

```
Conv(3→32)
ReLU
Pool
Conv(32→64)
ReLU
Pool
Conv(64→128)
ReLU
Pool
Flatten
FC(2048→512)
ReLU
Dropout
FC(512→256)
ReLU
Dropout
```

```
FC(256→classCount)
Softmax
```

这个结构比 LeNet 更强，加入了：

- 更多卷积核；
- 更深的特征提取；
- Dropout 防过拟合；
- 更大的全连接层。

汇报时可以说：

项目中提供了两种 CNN 架构：一种是 LeNet 风格的轻量网络，便于理解和训练；另一种是增强版网络，增加了卷积层数量、通道数和 Dropout，更适合复杂的交通标志分类任务。

CNN 的 forward

```
C++

Tensor output = input;
for (const auto& layer : layers_) {
    output = layer->forward(output);
}
```

这就是把输入依次传过每一层。

CNN 的 backward

```
C++

Tensor gradient = outputGradient;
for (auto iterator = layers_.rbegin(); iterator != layers_.rend(); ++iterator) {
    gradient = (*iterator)->backward(gradient);
}
```

反向传播和 forward 方向相反：

```
Softmax → FC → ReLU → Flatten → Pool → Conv
```

CNN 的 update

CNN 本身不直接更新参数，而是让每一层自己更新：

```
C++  
layer->update(...)
```

没有参数的层，比如 ReLU、Pooling、Flatten，默认什么都不做。

有参数的层，比如 Conv、FC，会更新自己的权重和偏置。

这个设计很清晰。

汇报时可以说：

CNN 类本身负责组织网络结构和传播流程，但具体计算由各层自己完成。这样每个层的职责清楚，网络结构也可以灵活扩展。

15. CNN.cpp 里的模型保存与读取

模型保存时，会写入：

```
magic 标识  
version  
classCount  
architecture  
trainableLayerCount  
每个可训练层的类型  
weights  
biases
```

其中 magic 是：

```
C++  
CPPCNN1
```

这是为了判断文件是不是本项目的模型文件。

它只保存可训练层，也就是：

```
ConvLayer  
FCLayer
```

不会保存：

```
ReLU
Pooling
Flatten
Softmax
Dropout
```

因为这些层没有可训练参数。

汇报时可以说：

模型文件只保存卷积层和全连接层的权重与偏置，因为这些才是训练得到的参数。激活层、池化层和展平层没有可训练参数，所以不需要保存。

16. `Trainer.cpp`：训练流程控制中心

这个文件是整个训练过程的总调度器。

训练流程大致是：

```
检查数据集
创建学习率调度器
创建数据增强器
划分训练集和验证集
读取 checkpoint
for 每个 epoch:
    构建训练顺序
    设置 network 为训练模式
    清空梯度
    for 每个样本:
        读取图片 Tensor
        数据增强
        forward 得到概率
        计算 loss
        统计 accuracy
        backward 反向传播
        如果 batch 结束:
            update 更新参数
    验证集评估
    保存 checkpoint
    记录 CSV 日志
    early stopping 判断
返回 TrainingReport
```

batch 训练逻辑

每个样本都会 backward:

```
C++  
network.backward(...)
```

梯度会累加在每一层的 `weightGradients_` 和 `biasGradients_` 中。

等到一个 batch 结束后:

```
C++  
gradientScale = 1.0F / currentBatchSize;  
network.update(...)
```

也就是说, 代码实现的是:

多个样本累计梯度, 然后除以 batch size, 再统一更新参数。

汇报时可以说:

训练器采用 mini-batch 梯度下降。每个 batch 中多个样本先累积梯度, batch 结束后再按平均梯度更新参数, 这比单样本更新更稳定。

类别均衡

`buildBalancedOrder()` 会统计每个类别的样本数量, 然后对少数类进行重复采样。

这对 GTSRB 这种类别不均衡的数据集很重要。

汇报时可以说:

为了缓解交通标志数据集中类别数量不均衡的问题, 训练代码实现了 class balancing, 通过过采样少数类别, 使模型不会过度偏向样本多的类别。

验证集与 early stopping

训练时可以划分验证集:

```
C++  
validationRatio
```

每轮训练结束后进行验证。

如果验证准确率长期不提升，就触发：

```
C++
earlyStoppingPatience
```

汇报时可以说：

Early stopping 用于防止过拟合。如果模型在验证集上的表现连续多轮没有提升，就提前停止训练，避免继续记忆训练集噪声。

checkpoint

保存 checkpoint 时会保存两部分：

```
模型参数文件
.opt 优化器状态文件
```

`.opt` 里保存：

```
epoch
best validation accuracy
训练选项
momentum velocity buffers
```

所以中断后可以 resume。

汇报时可以说：

项目不仅能保存模型参数，还能保存优化器状态，因此可以从中断位置继续训练，而不是只能重新开始。

17. 整体汇报时可以这样讲

你可以直接这样组织汇报：

```
本项目是一个使用 C++ 从底层实现的轻量级 CNN 框架，没有依赖 PyTorch 等深度学习框架。

首先，项目用 Tensor 类统一表示图像、特征图和分类向量。Tensor 内部使用 vector<float> 连续存储。

其次，项目设计了抽象基类 Layer，规定所有神经网络层都必须实现 forward 和 backward。卷积层、池化层和全连接层都继承自 Layer。

在网络结构上，CNN 类把多个 Layer 按顺序存入 vector 中。前向传播时，输入 Tensor 依次经过每一层。
```

训练部分由 `Trainer` 负责，它完成数据加载、数据增强、batch 梯度累计、学习率调度、验证集评估、early stop 等。最终，模型可以保存为二进制文件，只保存卷积层和全连接层的权重、偏置，因为其他层没有可训练参数。

18. 老师可能问的问题，以及你可以怎么回答

问：为什么 `Layer` 没有 `cpp`？

答：

因为 `Layer` 是抽象基类，主要用于定义接口。它的核心函数是纯虚函数，没有具体实现；其他默认函数已经直接写在头文件类定义里，属于 `inline` 实现，所以不需要单独的 `cpp` 文件。

问：为什么要用 `std::unique_ptr<Layer>`？

答：

因为不同层的具体类型不同，比如 `ConvLayer`、`ReLULayer`、`FCLayer`，但它们都继承自 `Layer`。使用 `unique_ptr<Layer>` 可以把不同类型的层统一放进同一个 `vector` 中，并通过虚函数实现多态调用。同时 `unique_ptr` 可以自动管理内存，避免手动 `delete`。

问：为什么 `forward` 时有些层要缓存输入？

答：

因为反向传播需要 `forward` 阶段的信息。比如卷积层计算权重梯度时需要原始输入，全连接层也需要输入向量，ReLU 需要知道哪些位置原来是正数，池化层需要知道最大值位置。因此这些信息必须在 `forward` 时缓存下来，供 `backward` 使用。

问：为什么模型只保存卷积层和全连接层？

答：

因为只有卷积层和全连接层有可训练参数，也就是权重和偏置。ReLU、Pooling、Flatten、Softmax、Dropout 没有训练得到的权重，所以不需要保存。

问：训练和推理有什么区别？

答：

训练时会启用 Dropout 和数据增强，并且会进行 backward 和 update；推理时关闭 Dropout，只做 forward，输出预测类别和置信度。

问：这个项目体现了哪些 C++ 特点？

答：

主要体现了面向对象、多态、RAII、智能指针、异常处理、标准库容器、随机数生成、二进制文件读写等 C++ 特性。尤其是 Layer 抽象基类和 `vector<unique_ptr<Layer>>` 的设计，使网络结构具有较好的扩展性。

19. 一句话总结整个代码

你可以在汇报最后这样说：

总体来说，这个项目用 C++ 手写实现了一个完整的 CNN 训练与推理框架，从 Tensor 数据结构、各类神经网络层、前向传播、反向传播、参数更新，到数据增强、学习率调度、模型保存和训练恢复，都形成了一个较完整的闭环。

所谓的卷积核、卷积层的概念到底是什么样的？我其实理解还是不够深入。

包括前向、反向传播的具体概念、数学操作也给我稍微细讲一下，我需要规范我的知识框架

这个问题很关键：你现在缺的不是某一行代码语法，而是 CNN 的“计算图视角”。我会先用直观图像讲卷积核/卷积层，再把它转成数学式，最后对应到你项目里的 `forward()`、`backward()`、`update()`。

已思考 18s

你可以先建立一个总框架：

卷积神经网络不是“神秘地识别图片”，而是在不断做这件事：用很多小矩阵在图片上滑

动，提取局部特征；再根据损失函数告诉它哪些小矩阵应该怎么改。

下面我按你汇报最需要的知识框架讲。

一、卷积核到底是什么？

1. 最直观理解

卷积核，也叫 filter / kernel，本质上就是一个**很小的权重矩阵**。

比如一个 **3×3** 卷积核：

```
[ 1  0 -1
  1  0 -1
  1  0 -1 ]
```

它会在图片上滑动，每次覆盖图片的一小块区域，比如：

图片局部区域：

```
[ 10  20  30
  10  20  30
  10  20  30 ]
```

然后做“对应位置相乘再求和”：

```
1×10 + 0×20 + (-1)×30
+ 1×10 + 0×20 + (-1)×30
+ 1×10 + 0×20 + (-1)×30
= -60
```

这个结果就成为输出特征图上的一个点。

所以卷积核可以理解为：

一个小型特征检测器。

不同卷积核会学到不同特征，比如：

```
检测竖直边缘
检测水平边缘
检测斜线
检测颜色变化
检测圆形局部结构
检测交通标志边框
```

不过注意：这些卷积核不是人提前写好的，而是训练出来的。

二、卷积层是什么？

卷积层不是一个卷积核，而是一组卷积核。

比如你的项目里有类似结构：

```
Conv(3 → 32, kernel = 3×3)
```

这句话的意思是：

```
输入有 3 个通道：R、G、B  
输出有 32 个通道  
卷积核大小是 3×3
```

那它到底有多少个卷积核参数？

不是简单的 $32 \times 3 \times 3$ 。

因为输入是 RGB 三通道，所以每个输出通道都要同时看输入的 3 个通道。

一个输出通道对应的卷积核实际大小是：

```
3 × 3 × 3
```

也就是：

```
输入通道数 × 卷积核高度 × 卷积核宽度
```

如果输出 32 个通道，那么总参数量是：

```
32 × 3 × 3 × 3 + 32 个 bias
```

即：

```
864 个权重 + 32 个偏置
```

三、一个卷积层具体做了什么？

假设输入是一张 RGB 图片：

```
Input: 3 × 32 × 32
```

经过一个卷积层：

```
Conv(3 → 32, kernel = 3, stride = 1, padding = 1)
```

输出就是：

```
Output: 32 × 32 × 32
```

为什么输出通道变成 32？

因为这一层有 **32 组卷积核**。

每一组卷积核负责生成一张特征图：

```
第 1 组卷积核 → 第 1 张特征图  
第 2 组卷积核 → 第 2 张特征图  
...  
第 32 组卷积核 → 第 32 张特征图
```

所以：

```
一个卷积核组生成一个输出通道  
多个卷积核组生成多个输出通道  
多个输出通道合起来就是卷积层的输出
```

汇报时可以这样说：

卷积层由多个卷积核组成。每个卷积核在输入图像或特征图上滑动，对局部区域做加权求和，生成一张新的特征图。多个卷积核可以提取不同类型的局部特征，因此卷积层的输出通常是多通道特征图。

四、卷积的数学形式

你项目里的卷积层计算大致对应这个公式：

```

$$Y[o, i, j] = b[o] + \sum X[c, i \cdot \text{stride} + u - \text{padding}, j \cdot \text{stride} + v - \text{padding}] \times W[o, c, u]$$

```

其中：

符号

x

y

w

b

o

c

i, j

u, v

含义

输入特征图

输出特征图

卷积核权重

偏置

输出通道编号

输入通道编号

输出特征图上的位置

卷积核内部的位置

你不一定要在汇报里完整写这个公式，但你要能理解它在干什么：

输出某个点
= 对输入局部区域做加权求和
+ 一个偏置

它和全连接层很像，只是全连接层是：

每个输出连接所有输入

而卷积层是：

每个输出只连接输入的一小块局部区域

这就是 CNN 适合图像的原因。

五、卷积层为什么适合图像？

因为图像有两个特点。

1. 局部性

图片里的一个边缘、角点、纹理，通常只和附近像素有关。

比如识别交通标志时，某个局部可能是：

红色边框
白色背景
黑色箭头的一部分
数字的一段笔画

卷积核每次只看局部区域，非常适合捕捉这些局部特征。

2. 权重共享

同一个卷积核会在整张图上滑动。

这意味着：

同一个“边缘检测器”可以在图片左上角检测边缘，也可以在图片右下角检测边缘。

这叫**权重共享**。

它的好处是参数量大幅减少。

如果用全连接层直接处理 `32×32×3` 图片，参数很多。

但卷积层只需要少量卷积核参数，就能在整张图上重复使用。

汇报时可以说：

卷积层利用局部连接和权重共享，既能提取图像局部特征，又能显著减少参数量，因此比普通全连接网络更适合图像任务。

六、前向传播是什么？

前向传播就是：

输入数据从第一层开始，一层一层向后计算，最终得到预测结果。

在你的项目里，大概是：

```
输入图片 Tensor
↓
ConvLayer.forward()
↓
ReLU.forward()
↓
MaxPoolingLayer.forward()
↓
ConvLayer.forward()
```

```
↓  
ReLU.forward()  
↓  
MaxPoolingLayer.forward()  
↓  
FlattenLayer.forward()  
↓  
FCLayer.forward()  
↓  
SoftmaxLayer.forward()  
↓  
输出每一类的概率
```

比如交通标志分类，最后可能输出：

```
限速 30: 0.02  
限速 50: 0.91  
禁止通行: 0.04  
向左转: 0.03
```

模型预测概率最高的是“限速 50”。

七、前向传播中的每一层在干什么？

你可以这样理解：

```
Conv: 提取局部特征  
ReLU: 加入非线性  
Pooling: 压缩特征图，保留显著特征  
Flatten: 把三维特征图拉平成一维向量  
FC: 根据特征做分类  
Softmax: 把分类分数转成概率
```

以交通标志为例：

```
原始图片  
↓  
第一层卷积：学到边缘、颜色块、简单纹理  
↓  
第二层卷积：学到圆形边框、三角形边框、箭头局部  
↓  
更深层：组合成完整交通标志形状  
↓  
全连接层：判断属于哪一类交通标志
```

八、损失函数是什么？

前向传播得到预测，但模型一开始是乱猜的。

比如真实标签是：

```
限速 50
```

模型输出却是：

```
限速 30: 0.60  
限速 50: 0.10  
禁止通行: 0.20  
向左转: 0.10
```

这就错了。

损失函数的作用是：

衡量模型预测和真实答案差得有多远。

你的项目用的是交叉熵损失：

```
Loss = -log(正确类别的预测概率)
```

如果正确类别概率高：

```
p = 0.9  
Loss = -log(0.9)  
损失小
```

如果正确类别概率低：

```
p = 0.1  
Loss = -log(0.1)  
损失大
```

所以交叉熵的核心思想是：

模型越不相信正确答案，惩罚越大。

九、反向传播是什么？

反向传播不是“重新跑一遍网络”，而是：

从损失函数开始，反过来计算每个参数对损失的影响。

更直白地说：

这个权重变大一点，Loss 会变大还是变小？
这个 bias 变大一点，Loss 会变大还是变小？
这个卷积核里的某个数应该增加还是减少？

反向传播得到的结果叫**梯度**。

梯度表示：

参数变化时，损失函数变化的方向和速度。

比如某个权重 w 的梯度是正的：

$$\partial \text{Loss} / \partial w > 0$$

说明：

w 增大，Loss 也增大

所以我们应该让 w 变小。

如果梯度是负的：

$$\partial \text{Loss} / \partial w < 0$$

说明：

w 增大，Loss 反而减小

所以我们应该让 w 变大。

最终参数更新规则就是：

$$w = w - \text{learning_rate} \times \text{gradient}$$

这就是梯度下降。

十、用一个极简例子理解反向传播

假设有一个超级简单模型：

$$y = w \times x + b$$

真实答案是：

$$\text{target} = 10$$

模型输出是：

$$y = 6$$

损失可以简单理解为：

$$\text{Loss} = (y - \text{target})^2$$

现在：

$$\text{Loss} = (6 - 10)^2 = 16$$

模型要知道：

w 应该变大还是变小？

b 应该变大还是变小？

因为输出 $y = 6$ 太小，而目标是 10 ，所以模型应该让输出变大。

如果输入 x 是正数，那么增大 w 会让 y 变大。

所以训练会推动 w 增大。

这就是反向传播的直觉：

先看预测错在哪里，再倒推每个参数应该怎么改。

十一、卷积层的反向传播具体算什么？

卷积层 forward 是：

$$\text{输出} = \text{输入局部区域} \times \text{卷积核权重} + \text{bias}$$

反向传播时要算三类梯度。

1. 对 bias 的梯度

bias 是直接加到输出上的：

$$Y = \dots + b$$

所以输出梯度会直接累加到 bias 上。

你代码里对应的是：

C++

```
biasGradients_[outputChannel] += gradient;
```

意思是：

这个输出通道上所有位置的误差，都会影响这个通道的 bias。

2. 对卷积核权重的梯度

某个卷积核权重参与了这样的计算：

$$Y = X \times W$$

所以：

$$W \text{ 的梯度} = \text{输入值 } X \times \text{输出梯度}$$

你代码里对应的是：

C++

```
weightGradients_[weightIndex] += cachedInput_.at(...) * gradient;
```

也就是说：

如果某个输入像素值很大，而且当前位置输出误差也很大，那么对应卷积核权重就会得到较大的调整。

3. 对输入的梯度

反向传播还要继续往前传，所以卷积层还要算：

输入 X 对 $Loss$ 的影响

因为：

$$Y = X \times W$$

所以：

$$X \text{ 的梯度} = W \times \text{输出梯度}$$

你代码里对应的是：

```
C++  
  
inputGradient.at(...) += weights_[weightIndex] * gradient;
```

这表示：

当前层要把误差信息继续传回上一层，让前面的层也能更新自己的参数。

十二、把卷积层 forward/backward 串起来理解

对于某个卷积层来说：

forward 阶段

```
输入 X  
卷积核 W  
偏置 b  
  
计算：  
Y = Conv(X, W) + b
```

同时缓存输入：

```
cachedInput = X
```

因为 backward 要用。

backward 阶段

从后一层传回来：

```
outputGradient = ∂Loss / ∂Y
```

然后算：

```
weightGradient = ∂Loss / ∂W  
biasGradient   = ∂Loss / ∂b  
inputGradient  = ∂Loss / ∂X
```

最后返回：

```
inputGradient
```

传给前一层。

十三、为什么 forward 要缓存输入？

因为反向传播需要它。

比如卷积核权重梯度：

```
weightGradient = 输入值 × 输出梯度
```

如果 forward 时不保存输入，那么 backward 时就不知道当时输入是什么了。

所以你的代码里有：

```
C++  
  
cachedInput_ = input;
```

这不是多余的，而是反向传播必须要用的。

类似地：

```
ReLU 要保存哪些位置 > 0  
Pooling 要保存最大值的位置  
FC 要保存输入向量  
Softmax 要保存概率分布
```

这就是神经网络训练中常见的：

forward 保存中间状态，backward 使用中间状态计算梯度。

十四、反向传播和链式法则的关系

反向传播的数学基础就是链式法则。

比如网络是：

```
输入 x
↓
第一层 a = f(x)
↓
第二层 b = g(a)
↓
损失 L = h(b)
```

如果要知道 x 对 L 的影响，就要算：

$$\partial L / \partial x = \partial L / \partial b \times \partial b / \partial a \times \partial a / \partial x$$

这就是为什么梯度要从后往前传。

神经网络每一层的 backward 本质上都在做：

```
拿到后一层传来的梯度
×
乘上本层自己的导数
=
传给前一层的梯度
```

所以你可以这样理解：

```
forward: 计算结果
backward: 计算责任
update: 根据责任修改参数
```

这个说法很适合汇报。

十五、ReLU 的反向传播

ReLU forward 是：

$$y = \max(0, x)$$

如果：

```
x > 0
```

那么：

```
y = x
```

梯度正常传递。

如果：

```
x <= 0
```

那么：

```
y = 0
```

梯度不传递。

所以 ReLU backward 是：

```
如果 forward 时 x > 0:  
    inputGradient = outputGradient  
否则:  
    inputGradient = 0
```

你项目里用 `active_` 记录哪些位置是正数：

```
C++  
  
active_[index] = input[index] > 0.0F ? 1U : 0U;
```

backward 时：

```
C++  
  
inputGradient[index] = active_[index] ? outputGradient[index] : 0;
```

汇报说法：

ReLU 在 forward 中保留正值、截断负值；在 backward 中，只有 forward 时被激活的位置才允许梯度继续传递。

十六、Pooling 的反向传播

最大池化 forward 是：

从一个局部窗口里取最大值

例如：

```
[ 1  3  
  2  5 ]
```

最大值是：

5

输出就是 5。

backward 时，梯度只传给当时的最大值位置：

```
[ 0  0  
  0 gradient ]
```

因为 forward 阶段只有 5 真正影响了输出，其他值没有影响输出。

所以你项目里会保存：

```
C++  
  
maximumIndices_
```

汇报说法：

最大池化层没有可训练参数，但它会改变梯度的流向。反向传播时，梯度只回传到 forward 阶段被选为最大值的位置。

十七、全连接层的反向传播

全连接层就是：

$$Y = W \times X + b$$

它和卷积层非常像，只是它连接的是整个向量，而不是局部窗口。

反向传播：

```
W 的梯度 = X × outputGradient  
b 的梯度 = outputGradient  
X 的梯度 = W × outputGradient
```

你项目里对应：

C++

```
weightGradients_[rowOffset + inputIndex] += cachedInput_[inputIndex] * gradient;  
biasGradients_[outputIndex] += gradient;  
inputGradient[inputIndex] += weights_[rowOffset + inputIndex] * gradient;
```

这和卷积层的思想完全一致。

十八、Softmax 和交叉熵

最后一层一般是 Softmax。

它把一组分数变成概率。

比如原始输出是：

```
[2.0, 1.0, 0.1]
```

Softmax 后可能是：

```
[0.66, 0.24, 0.10]
```

每一项都大于 0，而且加起来等于 1。

交叉熵损失只关心正确类别的概率：

```
Loss = -log(p_correct)
```

如果正确类别概率越大，Loss 越小。

十九、一次完整训练到底发生了什么？

假设训练一张图片。

第一步：forward

输入图片

↓

一层层计算

↓

得到分类概率

例如：

真实标签：限速 50

模型输出：

限速 30: 0.60

限速 50: 0.10

禁止通行: 0.20

向左转: 0.10

第二步：计算 Loss

```
Loss = -log(0.10)
```

因为正确类别概率太低，所以损失很大。

第三步：backward

从 Loss 开始反推：

Softmax backward

↓

FC backward

↓

ReLU backward

↓

Flatten backward

↓

Pooling backward

↓

Conv backward

每一层都计算：

自己的参数梯度

传给前一层的输入梯度

第四步：update

根据梯度更新参数：

```
weight = weight - learningRate × weightGradient  
bias    = bias    - learningRate × biasGradient
```

如果用了 Momentum，就是再加上“惯性”：

```
velocity = momentum × velocity + learningRate × gradient  
weight = weight - velocity
```

你项目里 ConvLayer 和 FCLayer 都有：

```
C++  
  
update()  
updateWithMomentum()
```

二十、你可以用这套话规范汇报

这一段可以直接背：

在本项目中，卷积核是卷积层中的可训练权重矩阵，它在输入图像或特征图上滑动，对局部区域进行加权求和，从而提取局部特征。一个卷积层通常包含多个卷积核组，每个卷积核组生成一个输出通道，因此卷积层可以同时提取多种不同特征。

前向传播时，输入 Tensor 依次经过卷积、激活、池化、展平、全连接和 Softmax 层，最终得到每个类别的预测概率。损失函数根据预测概率和真实标签计算误差。

反向传播时，网络从损失函数开始，利用链式法则从后向前计算每一层的梯度。对于卷积层来说，反向传播会计算卷积核权重梯度、偏置梯度，以及传回上一层的输入梯度。最后，优化器根据这些梯度更新卷积核和全连接层的参数，使模型在下一次预测中更接近真实标签。

二十一、最核心的知识框架

你可以把 CNN 训练理解成这四句话：

1. 卷积核是局部特征检测器。
2. 前向传播是用当前参数做一次预测。

3. 损失函数衡量预测错得多严重。
4. 反向传播计算每个参数该怎么改，优化器真正修改参数。

再进一步：

Conv forward:

用卷积核提取特征。

Conv backward:

算卷积核对错误的责任。

Update:

根据责任修改卷积核。

你把这三层理解打通，CNN 的主体就清楚了。

我这次会回到你上传的代码本身，不再扩展太多理论。重点按文件讲：这个文件在工程里负责什么、它和其他文件怎么连接、训练时它什么时候被调用。

已思考 32s

对，回到你的代码本身，这个工程可以理解成一个**手写 CNN 小框架**。它不是只有“一个模型”，而是把深度学习训练流程拆成了很多模块。

整体关系是：

```
Tensor
↓
Layer 抽象接口
↓
Conv / ReLU / Pool / Flatten / FC / Dropout / Softmax
↓
CNN 把这些 Layer 串起来
↓
Loss 计算损失
↓
Trainer 控制训练流程
↓
Augmenter / LRScheduler / Checkpoint 辅助训练
```

下面按文件讲。

1. `Tensor.cpp`：统一的数据容器

这个文件实现的是 `Tensor` 类。

你的整个 CNN 里，图片、特征图、全连接层输入输出、Softmax 概率，本质上都用 `Tensor` 表示。

它内部有三个维度：

```
C++  
  
channels_  
height_  
width_
```

以及真正存数据的：

```
C++  
  
std::vector<float> values_;
```

也就是说，一个 `3 × 32 × 32` RGB 图片在代码里就是：

```
channels = 3  
height = 32  
width = 32  
values_ = 3072 个 float
```

它的作用主要有：

1. 保存三维数据
2. 提供 `at(channel, row, column)` 访问接口
3. 提供 `operator[]` 按一维下标访问
4. 负责 `resize` / `fill` / `size` / `empty`
5. 检查越界和维度溢出

关键点是这个函数：

```
C++  
  
return (channel * height_ + row) * width_ + column;
```

它把三维坐标：

```
(channel, row, column)
```

转成一维数组下标。

所以 `Tensor` 是整个工程的数据基础。

2. `Layer.h`：所有网络层的统一接口

这个是抽象基类。

它规定所有层都必须有：

C++

```
virtual Tensor forward(const Tensor& input) = 0;  
virtual Tensor backward(const Tensor& outputGradient) = 0;  
virtual std::string type() const = 0;
```

也就是说，不管是卷积层、池化层、ReLU 层、全连接层，都必须实现：

forward: 前向传播
backward: 反向传播
type: 返回层类型

另外它还给了一些默认函数：

C++

```
zeroGrad()  
update()  
updateWithMomentum()  
train()  
isTrainable()  
saveOptimizerState()  
loadOptimizerState()
```

这些默认什么也不做。

为什么？

因为不是所有层都有参数。

比如：

ConvLayer: 有参数，要 update
FCLayer: 有参数，要 update
ReLU: 没参数，不需要 update
Pooling: 没参数，不需要 update
Flatten: 没参数，不需要 update
Softmax: 没参数，不需要 update
Dropout: 没参数，但需要区分训练/推理模式

所以 `Layer.h` 的作用是：

给所有层提供统一接口，让 `CNN.cpp` 可以用一个 `vector<unique_ptr<Layer>>` 管理所有层。

3. `ConvLayer.cpp`：卷积层

这是 CNN 里最核心的层。

它负责实现：

- 卷积前向传播
- 卷积反向传播
- 卷积核参数更新
- Momentum 更新
- 梯度清零
- 优化器状态保存/读取

构造函数里会创建这些数据：

```
C++  
  
weights_  
biases_  
weightGradients_  
biasGradients_  
weightVelocity_  
biasVelocity_
```

含义是：

- `weights_`：卷积核权重
- `biases_`：每个输出通道的偏置
- `weightGradients_`：卷积核梯度
- `biasGradients_`：偏置梯度
- `weightVelocity_`：Momentum 用的权重速度
- `biasVelocity_`：Momentum 用的偏置速度

卷积核参数数量是：

```
C++  
  
outputChannels_ * inputChannels_ * kernelSize_ * kernelSize_
```

比如：

```
ConvLayer(3, 32, 5, 1, 2)
```

输入通道 3

输出通道 32

卷积核 5×5

权重数量 = $32 \times 3 \times 5 \times 5 = 2400$

bias 数量 = 32

forward()

它做的事是：

用卷积核扫描输入 Tensor

对局部区域加权求和

加 bias

生成输出特征图

核心逻辑是：

C++

```
sum += input.at(inputChannel, inputRow, inputColumn)
        * weights_[weightOffset(...)]
```

它也处理了 padding：

C++

```
if (inputRow < 0 || inputColumn < 0
    || inputRow >= input.height()
    || inputColumn >= input.width()) {
    continue;
}
```

也就是说，padding 区域相当于补 0。

同时它会保存输入：

C++

```
cachedInput_ = input;
```

这是为了反向传播时计算梯度。

`backward()`

它做三件事：

第一，算 bias 梯度：

C++

```
biasGradients_[outputChannel] += gradient;
```

第二，算卷积核权重梯度：

C++

```
weightGradients_[weightIndex] += inputValue * gradient;
```

第三，把梯度传回上一层：

C++

```
inputGradient.at(... ) += weights_[weightIndex] * gradient;
```

所以卷积层反向传播既要：

更新自己

也要：

把误差信息传给前面的层

`update()` 和 `updateWithMomentum()`

`update()` 是普通 SGD：

C++

```
weights_[index] -= learningRate * gradient;
```

`updateWithMomentum()` 是 Momentum SGD：

C++

```
weightVelocity_[index] = momentum * weightVelocity_[index]  
                        + learningRate * gradient;  
weights_[index] -= weightVelocity_[index];
```

这里 `weightVelocity_` 保存的是历史更新趋势。

4. `Activation.cpp`: ReLU 和 Softmax

这个文件里有两个层：

ReLU Layer
Softmax Layer

ReLU Layer

前向传播：

C++

```
output[index] = std::max(0.0F, input[index]);
```

也就是：

负数变 0
正数保留

它还保存了一个 `active_`：

C++

```
active_[index] = input[index] > 0.0F ? 1U : 0U;
```

反向传播时：

C++

```
inputGradient[index] = active_[index] ? outputGradient[index] : 0.0F;
```

意思是：

forward 时大于 0 的位置，梯度继续传
forward 时小于等于 0 的位置，梯度变 0

所以 ReLU 文件负责给网络加入非线性。

SoftmaxLayer

前向传播：

```
C++  
  
probabilities_[index] = std::exp(input[index] - maximum);  
probability /= sum;
```

它把最后的类别分数变成概率。

这里减去 `maximum` 是为了数值稳定，避免 `exp()` 爆掉。

反向传播里实现的是 Softmax 的梯度传播。

在你的网络里，Softmax 是最后一层。

5. PoolingLayer.cpp：最大池化层

这个文件实现的是：

```
MaxPoolingLayer
```

前向传播时，它对每个通道单独做最大池化。

比如：

```
2x2 区域：  
1  3  
2  5  
  
输出：5
```

代码里会保存最大值的位置：

```
C++  
  
maximumIndices_
```

反向传播时：

```
C++  
  
inputGradient[maximumIndices_[index]] += outputGradient[index];
```

也就是说：

哪个位置在 forward 时被选为最大值，backward 时梯度就传回哪个位置。

它的作用是：

1. 降低特征图尺寸
2. 保留局部最强响应
3. 减少后续计算量

6. FlattenLayer.cpp：展平层

这个层没有参数。

前向传播：

C++

```
Tensor output(1, 1, input.size());  
output.values() = input.values();
```

比如：

$128 \times 4 \times 4$

变成：

$1 \times 1 \times 2048$

也就是把三维特征张量拉成一维向量。

反向传播时再恢复原来的形状：

C++

```
Tensor inputGradient(inputChannels_, inputHeight_, inputWidth_);  
inputGradient.values() = outputGradient.values();
```

所以它的功能很单纯：

连接卷积部分和全连接部分

卷积层输出是三维的，全连接层需要一维向量，所以中间需要 Flatten。

7. FCLayer.cpp：全连接层

这是普通神经网络层。

它实现：

```
output = W × input + b
```

构造函数里创建：

```
C++  
  
weights_  
biases_  
weightGradients_  
biasGradients_  
weightVelocity_  
biasVelocity_
```

和卷积层类似，只不过它的权重是二维矩阵逻辑。

比如：

```
FCLayer(2048, 512)
```

含义是：

```
输入 2048 个数字  
输出 512 个数字
```

参数数量：

```
2048 × 512 个权重  
512 个 bias
```

`forward()`

核心逻辑：

```
C++  
  
sum += weights_[rowOffset + inputIndex] * input[inputIndex];
```

每个输出神经元都看全部输入。

`backward()`

反向传播时计算：

C++

```
weightGradients_[...] += cachedInput_[inputIndex] * gradient;
biasGradients_[outputIndex] += gradient;
inputGradient[inputIndex] += weights_[...] * gradient;
```

所以 FC 层和 Conv 层一样，也是可训练层。

区别是：

ConvLayer：局部连接 + 权重共享
FCLayer：每个输出连接全部输入

8. `DropoutLayer.cpp`：Dropout 防过拟合层

这个层只在训练时启用。

构造函数里设置：

C++

```
rate_  
scale_  
generator_
```

比如：

```
DropoutLayer(0.5)
```

意思是训练时随机丢掉 50% 的神经元输出。

前向传播时，如果是推理模式：

C++

```
if (!training_) {  
    copy input to output;  
    return output;  
}
```

也就是不做任何处理。

如果是训练模式，就随机保留或丢弃：

C++

```
std::bernoulli_distribution distribution(1.0F - rate_);
```

保留的值会乘以：

C++

```
scale_ = 1.0F / (1.0F - rate_);
```

这是 inverted dropout。

比如 rate=0.5，那么保留下来的值乘以 2。

这样推理时就不需要再缩放。

反向传播时，被丢掉的位置梯度也是 0。

9. `Loss.cpp`：损失函数和指标

这个文件里不是网络层，而是训练评估工具。

主要包括：

```
CrossEntropyLoss  
argmax  
Accuracy  
PerClassAccuracy  
ConfusionMatrix
```

`CrossEntropyLoss`

计算损失：

C++

```
return -std::log(std::max(probabilities[label], 1.0e-7F));
```

如果正确类别概率越高，loss 越小。

如果正确类别概率越低，loss 越大。

梯度：

C++

```
result[label] = -1.0F / std::max(probabilities[label], 1.0e-7F);
```

这里生成的是对 Softmax 输出概率的梯度，然后交给 `network.backward()`。

argmax

找最大概率对应的类别：

C++

```
std::max_element(...)
```

用于预测类别。

Accuracy

统计整体准确率。

```
预测类别 == 真实类别
```

```
correct++
```

PerClassAccuracy

统计每个类别自己的准确率。

这个对交通标志数据集有意义，因为某些类别样本多，某些类别样本少。

ConfusionMatrix

混淆矩阵。

记录：

```
模型把真实类别 A 预测成类别 B 的次数
```

可以用来查看哪些类别容易混淆。

10. LRScheduler.cpp：学习率调度器

这个文件控制每个 epoch 用多大学习率。

支持：

```
Warmup  
Step  
MultiStep  
Cosine
```

虽然代码里 `Step` 和 `MultiStep` 实际逻辑差不多，都是按 `stepSize` 间隔衰减。

Warmup

如果设置了 warmup：

```
C++  
  
return minLR + (initialLR - minLR) * progress;
```

意思是训练开始几轮学习率从小慢慢升高。

Step / MultiStep

```
C++  
  
lr *= pow(decayFactor, effectiveEpoch / stepSize);
```

每隔一定 epoch，学习率乘以衰减系数。

Cosine

```
C++  
  
lr = minLR + (initialLR - minLR) * cosine;
```

用余弦曲线慢慢降低学习率。

它的作用是：

前期步子大一点，方便快速下降
后期步子小一点，方便稳定收敛

11. Augmenter.cpp：数据增强

这个文件用于训练时随机改造图片，让模型见到更多变化。

它支持：

旋转
平移
缩放
亮度变化
对比度变化
噪声

主入口是：

C++

```
Tensor Augmenter::augment(const Tensor& input)
```

内部按配置依次调用：

C++

```
applyRotation()  
applyTranslation()  
applyScaling()  
applyBrightness()  
applyContrast()  
applyNoise()
```

旋转和缩放用到了：

C++

```
sampleChannel()
```

这是双线性插值。

因为旋转、缩放之后，目标像素对应到原图的位置可能不是整数坐标，所以要根据周围像素插值。

最后会把像素限制在 `[0, 1]`:

C++

```
val = std::max(0.0F, std::min(1.0F, val));
```

这个文件的作用是:

提高泛化能力, 减少模型只记住训练图片的风险

12. `CNN.cpp`: 网络本体

这个文件是把所有层组装起来的地方。

它里面维护:

C++

```
std::vector<std::unique_ptr<Layer>> layers_;
```

所以 CNN 并不自己写每一层的计算, 而是把不同 Layer 串起来。

两种网络结构

你的代码里有两种架构。

LeNet 风格

C++

```
ConvLayer(3, 6, 5, 1, 0)
ReLU
MaxPoolingLayer(2, 2)
ConvLayer(6, 16, 5, 1, 0)
ReLU
MaxPoolingLayer(2, 2)
Flatten
FCLayer(16 * 5 * 5, 120)
ReLU
FCLayer(120, classCount)
Softmax
```

尺寸大概是：

```
输入: 3 × 32 × 32
Conv1: 6 × 28 × 28
Pool1: 6 × 14 × 14
Conv2: 16 × 10 × 10
Pool2: 16 × 5 × 5
Flatten: 400
FC: 120
FC: classCount
Softmax: classCount
```

Enhanced 架构

C++

```
ConvLayer(3, 32, 5, 1, 2)
ReLU
MaxPool

ConvLayer(32, 64, 3, 1, 1)
ReLU
MaxPool

ConvLayer(64, 128, 3, 1, 1)
ReLU
MaxPool

Flatten
FCLayer(128 * 4 * 4, 512)
ReLU
Dropout(0.5)

FCLayer(512, 256)
ReLU
Dropout(0.5)

FCLayer(256, classCount)
Softmax
```

尺寸大概是：

```
输入: 3 × 32 × 32
Conv1: 32 × 32 × 32
Pool1: 32 × 16 × 16
Conv2: 64 × 16 × 16
Pool2: 64 × 8 × 8
```

```
Conv3: 128 × 8 × 8  
Pool3: 128 × 4 × 4  
Flatten: 2048  
FC1: 512  
FC2: 256  
FC3: classCount  
Softmax: classCount
```

这个 Enhanced 架构更强，因为：

```
卷积通道更多  
卷积层更深  
FC 层更宽  
加了 Dropout
```

`forward()`

```
C++  
  
Tensor output = input;  
for (const auto& layer : layers_) {  
    output = layer->forward(output);  
}  
return output;
```

就是把输入依次传过所有层。

`backward()`

```
C++  
  
Tensor gradient = outputGradient;  
for (auto iterator = layers_.rbegin(); iterator != layers_.rend(); ++iterator) {  
    gradient = (*iterator)->backward(gradient);  
}  
return gradient;
```

就是从最后一层往前反向传播。

`update()` / `updateWithMomentum()`

CNN 自己不更新参数，而是通知每一层：

C++

```
layer->update(...)
```

有参数的层会更新。

没参数的层默认什么都不做。

`predict()`

C++

```
setTraining(false);  
const Tensor probabilities = forward(input);  
const std::size_t prediction = argmax(probabilities);
```

推理时关闭 Dropout，然后 forward，取最大概率类别。

`saveModel()` / `loadModel()`

保存和读取模型。

它只保存：

ConvLayer 的 weights 和 biases
FCLayer 的 weights 和 biases

不保存：

ReLU
Pooling
Flatten
Softmax
Dropout

因为这些层没有可训练参数。

模型文件里还写了：

```
magic 标识: CPPCNN1  
version  
classCount  
architecture  
trainableLayerCount
```

```
layer type
weights
biases
```

所以它能检查：

```
模型文件是不是本项目的
类别数是否匹配
网络结构是否匹配
参数数量是否匹配
```

13. `Trainer.cpp`：训练控制器

这个文件是整个训练流程的总指挥。

它不负责具体卷积计算，而是负责：

```
加载数据
划分训练集/验证集
数据增强
构造训练顺序
前向传播
计算 loss
反向传播
batch 更新
学习率调度
验证集评估
early stopping
保存 checkpoint
写 CSV 日志
```

```
train()
```

这是训练主函数。

核心流程是：

```
检查数据集
创建学习率调度器
创建数据增强器
划分训练集/验证集
尝试恢复 checkpoint
for epoch:
    设置学习率
    构造训练顺序
```

```
network.setTraining(true)
network.zeroGrad()
for 每个训练样本:
    读取图片
    可选数据增强
    network.forward()
    计算 loss
    统计 accuracy
    network.backward()
    累积 batch 梯度
    batch 满了就 update
做验证
保存 checkpoint
写 CSV
判断 early stopping
```

mini-batch 更新

代码里每张图片都会 backward:

```
C++

network.backward(...)
++currentBatchSize;
```

但是不是每张图都立刻 update。

而是 batch 满了之后:

```
C++

gradientScale = 1.0F / currentBatchSize;
network.update(...)
```

所以它实现的是 mini-batch SGD。

类别均衡

`buildBalancedOrder()` 会统计每类样本数，然后对少数类进行重复采样。

作用是：

避免模型过度偏向样本多的类别

验证集

如果设置了 `validationRatio`，它会拆分训练集和验证集。

支持两种方式：

```
按 track 划分  
随机划分
```

这里 `DataLoader::splitByTrack()` 在你上传的 cpp 里没有定义，应该在别的文件或头文件里。

`evaluate()` / `evaluateDetailed()`

`evaluate()` 做普通评估：

```
loss  
accuracy
```

`evaluateDetailed()` 还会计算：

```
per-class accuracy  
confusion matrix
```

checkpoint

`saveCheckpoint()` 会保存两部分：

```
模型权重文件  
.opt 文件
```

`.opt` 里保存：

```
epoch  
bestValAccuracy  
训练选项 flag  
optimizer state
```

optimizer state 主要就是 Momentum 的 velocity。

这样 resume 时不是只恢复权重，还能恢复 Momentum 状态。

14. 按一次训练过程看各文件怎么配合

假设训练一张图片。

第一步：DataLoader 读图

这个部分没有上传 cpp, 但 Trainer 调用了：

```
C++
```

```
loader.loadTensor(sample)
```

它应该负责把图片转成：

```
Tensor(3, 32, 32)
```

第二步：Augmenter 可选增强

```
C++
```

```
input = augmenter.augment(input);
```

调用的是：

```
Augmenter.cpp
```

第三步：CNN forward

```
C++
```

```
network.forward(input)
```

调用：

```
CNN.cpp
```

CNN 依次调用：

```
ConvLayer.cpp  
Activation.cpp
```

```
PoolingLayer.cpp  
FlattenLayer.cpp  
FCLayer.cpp  
DropoutLayer.cpp  
SoftmaxLayer
```

第四步：Loss 计算误差

C++

```
CrossEntropyLoss::value(probabilities, label)
```

调用：

```
Loss.cpp
```

第五步：Loss 生成梯度

C++

```
CrossEntropyLoss::gradient(probabilities, label)
```

还是：

```
Loss.cpp
```

第六步：CNN backward

C++

```
network.backward(...)
```

调用顺序反过来：

```
Softmax  
FC  
Dropout  
ReLU  
FC  
Flatten
```

```
Pooling
ReLU
Conv
...
```

对应实现分散在：

```
Activation.cpp
FCLayer.cpp
DropoutLayer.cpp
FlattenLayer.cpp
PoolingLayer.cpp
ConvLayer.cpp
```

第七步：参数更新

batch 结束后：

```
C++
network.update(...)
```

或者：

```
C++
network.updateWithMomentum(...)
```

真正会更新参数的是：

```
ConvLayer.cpp
FCLayer.cpp
```

其他层没有参数，不更新。

15. 哪些文件属于“模型结构”，哪些属于“训练系统”？

模型结构核心：

```
Tensor.cpp
Layer.h
ConvLayer.cpp
Activation.cpp
PoolingLayer.cpp
```

```
FlattenLayer.cpp  
FCLayer.cpp  
DropoutLayer.cpp  
CNN.cpp
```

训练系统核心：

```
Loss.cpp  
Trainer.cpp  
LRScheduler.cpp  
Augmenter.cpp
```

保存/恢复相关：

```
CNN.cpp: 保存模型权重  
Trainer.cpp: 保存 checkpoint 和 optimizer 状态  
ConvLayer.cpp / FCLayer.cpp: 保存 Momentum velocity
```

16. 哪些层有可训练参数？

有可训练参数：

```
ConvLayer  
FCLayer
```

没有可训练参数：

```
ReLU  
Softmax  
MaxPooling  
Flatten  
Dropout
```

但注意：

```
Dropout 没有参数，但有训练/推理模式差异。  
ReLU 没有参数，但会缓存 active mask。  
Pooling 没有参数，但会缓存最大值位置。  
Flatten 没有参数，但会缓存原始形状。  
Softmax 没有参数，但会缓存概率。
```

这些缓存是为了 backward。

17. 你这个代码最核心的主线

一句话就是：

```
Tensor 负责装数据；
Layer 规定每层必须 forward/backward；
Conv/FC 是可训练层；
ReLU/Pool/Flatten/Dropout/Softmax 是功能层；
CNN 把层串起来；
Loss 告诉模型错得多严重；
Trainer 负责一轮一轮训练；
Augmenter 增强数据；
LRScheduler 调整学习率；
save/load 和 checkpoint 负责保存训练成果。
```

这个代码整体逻辑是清楚的：

前面是自研神经网络层，后面是训练管理系统。

那我这个项目真正使用的时候各部分的参数都是多少？包括卷积核大小、通道数、stride、padding、各层规模等等。还有，全连接层，也就是FC1和2之间的连接，用的也是ReLU函数吗？

已思考 26s

是的，回到你的代码看，**网络结构不是动态自动生成的，而是在 `CNN.cpp` 的构造函数里写死了两套架构：**

```
C++
```

```
CNN::CNN(classCount, seed, arch)
```

也就是说，真正使用哪一套，取决于创建 `CNN` 对象时传入的 `arch` 是：

```
C++
```

```
CNNArchitecture::LeNet
```

还是：

```
C++
```

我没看到你项目里的 `main.cpp` 或调用代码，所以不能断言你运行时一定用哪一个；但从 `CNN.cpp` 看，支持下面两套。

一、输入固定是多少？

你的 `CNN::forward()` 明确要求输入必须是：

`3 x 32 x 32`

也就是：

3 个通道：RGB

高度：32

宽度：32

如果不是这个尺寸，代码会直接报错：

C++

```
if (input.channels() != 3 || input.height() != 32 || input.width() != 32) {  
    throw std::invalid_argument("CNN expects a normalized 3 x 32 x 32 input tensor.");  
}
```

所以你的项目默认处理的是：

归一化后的 32x32 RGB 图片

二、LeNet 架构参数

代码里 LeNet 是这一段：

C++

```
ConvLayer(3, 6, 5, 1, 0)  
ReLU  
MaxPoolingLayer(2, 2)  
  
ConvLayer(6, 16, 5, 1, 0)  
ReLU  
MaxPoolingLayer(2, 2)
```

```
Flatten
FCLayer(16 * 5 * 5, 120)
ReLU
FCLayer(120, classCount)
Softmax
```

LeNet 尺寸变化

输入：
 $3 \times 32 \times 32$

Conv1

```
C++
ConvLayer(3, 6, 5, 1, 0)
```

含义是：

输入通道数：3
输出通道数：6
卷积核大小：5×5
stride: 1
padding: 0

因为 padding=0，所以空间尺寸会变小。

输出尺寸：

$6 \times 28 \times 28$

计算：

$(32 + 2 \times 0 - 5) / 1 + 1 = 28$

参数量：

$6 \times 3 \times 5 \times 5 + 6 = 456$

ReLU1

```
6 × 28 × 28 → 6 × 28 × 28
```

只改变数值，不改变形状。

Pool1

```
C++
```

```
MaxPoolingLayer(2, 2)
```

含义是：

```
池化窗口：2×2  
stride: 2
```

输出：

```
6 × 14 × 14
```

Conv2

```
C++
```

```
ConvLayer(6, 16, 5, 1, 0)
```

含义是：

```
输入通道数：6  
输出通道数：16  
卷积核大小：5×5  
stride: 1  
padding: 0
```

注意这里每个卷积核实际是：

```
6 × 5 × 5
```

输出：

$$16 \times 10 \times 10$$

计算：

$$(14 + 2 \times 0 - 5) / 1 + 1 = 10$$

参数量：

$$16 \times 6 \times 5 \times 5 + 16 = 2416$$

ReLU2

$$16 \times 10 \times 10 \rightarrow 16 \times 10 \times 10$$

Pool2

$$16 \times 10 \times 10 \rightarrow 16 \times 5 \times 5$$

Flatten

$$16 \times 5 \times 5 \rightarrow 400$$

因为：

$$16 \times 5 \times 5 = 400$$

FC1

C++

```
FCLayer(16 * 5 * 5, 120)
```

也就是：

```
400 → 120
```

参数量：

```
400 × 120 + 120 = 48120
```

ReLU3

这里有 ReLU。

```
120 → 120
```

所以 LeNet 里，FC1 和最后一个 FC 之间确实用了 ReLU。

FC2

```
C++
```

```
FCLayer(120, classCount)
```

也就是：

```
120 → classCount
```

如果你用的是 GTSRB 交通标志数据集，通常 `classCount = 43`，那么就是：

```
120 → 43
```

参数量：

```
120 × 43 + 43 = 5203
```

Softmax

```
classCount 个分数 → classCount 个概率
```

如果 `classCount = 43`：

43 个 logits → 43 个概率

LeNet 总结构

```
3 × 32 × 32
↓ Conv 5×5, 3→6, stride=1, padding=0
6 × 28 × 28
↓ ReLU
6 × 28 × 28
↓ MaxPool 2×2, stride=2
6 × 14 × 14
↓ Conv 5×5, 6→16, stride=1, padding=0
16 × 10 × 10
↓ ReLU
16 × 10 × 10
↓ MaxPool 2×2, stride=2
16 × 5 × 5
↓ Flatten
400
↓ FC 400→120
120
↓ ReLU
120
↓ FC 120→classCount
classCount
↓ Softmax
classCount 个概率
```

如果 `classCount=43`，总参数量大约是：

56195

三、Enhanced 架构参数

代码里 Enhanced 是这一段：

```
C++

ConvLayer(3, 32, 5, 1, 2)
ReLU
MaxPoolingLayer(2, 2)

ConvLayer(32, 64, 3, 1, 1)
ReLU
```

```

MaxPoolingLayer(2, 2)

ConvLayer(64, 128, 3, 1, 1)
ReLU
MaxPoolingLayer(2, 2)

Flatten

FCLayer(128 * 4 * 4, 512)
ReLU
DropoutLayer(0.5)

FCLayer(512, 256)
ReLU
DropoutLayer(0.5)

FCLayer(256, classCount)
Softmax

```

这套比 LeNet 明显更强。

Enhanced 尺寸变化

输入：

3 × 32 × 32

Conv1

C++

```
ConvLayer(3, 32, 5, 1, 2)
```

含义是：

```

输入通道数: 3
输出通道数: 32
卷积核大小: 5×5
stride: 1
padding: 2

```

因为 5×5 卷积核配 padding=2，空间尺寸保持不变。

输出：

```
32 × 32 × 32
```

计算：

```
(32 + 2×2 - 5) / 1 + 1 = 32
```

每个卷积核实际大小：

```
3 × 5 × 5
```

参数量：

```
32 × 3 × 5 × 5 + 32 = 2432
```

ReLU1

```
32 × 32 × 32 → 32 × 32 × 32
```

Pool1

```
C++
```

```
MaxPoolingLayer(2, 2)
```

输出：

```
32 × 16 × 16
```

Conv2

```
C++
```

```
ConvLayer(32, 64, 3, 1, 1)
```

含义是：

```
输入通道数：32
```

```
输出通道数：64
```

```
卷积核大小: 3×3  
stride: 1  
padding: 1
```

因为 3×3 配 `padding=1`，空间尺寸保持不变。

输出：

```
64 × 16 × 16
```

每个卷积核实际大小：

```
32 × 3 × 3
```

参数量：

```
64 × 32 × 3 × 3 + 64 = 18496
```

ReLU2

```
64 × 16 × 16 → 64 × 16 × 16
```

Pool2

```
64 × 16 × 16 → 64 × 8 × 8
```

Conv3

```
C++
```

```
ConvLayer(64, 128, 3, 1, 1)
```

含义是：

```
输入通道数: 64  
输出通道数: 128  
卷积核大小: 3×3
```

```
stride: 1  
padding: 1
```

输出:

```
128 × 8 × 8
```

每个卷积核实际大小:

```
64 × 3 × 3
```

参数量:

```
128 × 64 × 3 × 3 + 128 = 73856
```

ReLU3

```
128 × 8 × 8 → 128 × 8 × 8
```

Pool3

```
128 × 8 × 8 → 128 × 4 × 4
```

Flatten

```
128 × 4 × 4 → 2048
```

因为:

```
128 × 4 × 4 = 2048
```

FC1

```
C++
```

```
FCLayer(128 * 4 * 4, 512)
```

也就是：

```
2048 → 512
```

参数量：

```
2048 × 512 + 512 = 1049088
```

ReLU4

这里有 ReLU：

```
512 → 512
```

Dropout1

```
C++
```

```
DropoutLayer(0.5F)
```

训练时随机丢弃 50% 的神经元输出。

输出尺寸不变：

```
512 → 512
```

FC2

```
C++
```

```
FCLayer(512, 256)
```

也就是：

```
512 → 256
```

参数量：

$$512 \times 256 + 256 = 131328$$

ReLU5

这里也有 ReLU：

$$256 \rightarrow 256$$

Dropout2

C++

```
DropoutLayer(0.5F)
```

输出尺寸不变：

$$256 \rightarrow 256$$

FC3

C++

```
FCLayer(256, classCount)
```

注意：Enhanced 不是只有 FC1、FC2 两个全连接层，而是有 **三个全连接层**：

```
FC1: 2048 → 512  
FC2: 512 → 256  
FC3: 256 → classCount
```

如果 `classCount=43`：

$$256 \rightarrow 43$$

参数量：

$$256 \times 43 + 43 = 11051$$

Softmax

classCount 个分数 $\rightarrow$ classCount 个概率

如果是 43 类:

43 个 logits $\rightarrow$ 43 个概率

Enhanced 总结构

```
3 × 32 × 32
↓ Conv 5×5, 3→32, stride=1, padding=2
32 × 32 × 32
↓ ReLU
32 × 32 × 32
↓ MaxPool 2×2, stride=2
32 × 16 × 16
↓ Conv 3×3, 32→64, stride=1, padding=1
64 × 16 × 16
↓ ReLU
64 × 16 × 16
↓ MaxPool 2×2, stride=2
64 × 8 × 8
↓ Conv 3×3, 64→128, stride=1, padding=1
128 × 8 × 8
↓ ReLU
128 × 8 × 8
↓ MaxPool 2×2, stride=2
128 × 4 × 4
↓ Flatten
2048
↓ FC 2048→512
512
↓ ReLU
512
↓ Dropout 0.5
512
↓ FC 512→256
256
↓ ReLU
256
↓ Dropout 0.5
256
↓ FC 256→classCount
classCount
```

```
↓ Softmax  
classCount 个概率
```

如果 `classCount=43`，总参数量大约是：

```
1286251
```

也就是约 128.6 万参数。

四、你问：FC1 和 FC2 之间用的是 ReLU 吗？

答案：是的。

但要分架构说。

如果是 LeNet

LeNet 里全连接部分是：

```
Flatten  
↓  
FC 400→120  
↓  
ReLU  
↓  
FC 120→classCount  
↓  
Softmax
```

所以：

```
FC1 和 FC2 之间有 ReLU
```

没有 Dropout。

如果是 Enhanced

Enhanced 里全连接部分是：

```
Flatten  
↓  
FC 2048→512  
↓
```

```
ReLU
↓
Dropout 0.5
↓
FC 512→256
↓
ReLU
↓
Dropout 0.5
↓
FC 256→classCount
↓
Softmax
```

所以更准确地说：

```
FC1 和 FC2 之间：
ReLU + Dropout

FC2 和 FC3 之间：
ReLU + Dropout

最后 FC3 和 Softmax 之间：
没有 ReLU
```

最后一层不能再接 ReLU，因为最后要保留每个类别的原始分数，也就是 logits，然后交给 Softmax 做概率归一化。

五、两套结构对比

项目	LeNet	Enhanced
输入	3×32×32	3×32×32
卷积层数	2	3
卷积通道	3→6→16	3→32→64→128
卷积核	5×5, 5×5	5×5, 3×3, 3×3
padding	0, 0	2, 1, 1
stride	全部卷积都是 1	全部卷积都是 1
池化	两次 2×2, stride=2	三次 2×2, stride=2

项目	LeNet	Enhanced
Flatten 后	400	2048
全连接	400→120→classCount	2048→512→256→classCount
Dropout	无	两个 0.5
如果 43 类参数量	约 5.6 万	约 128.6 万

你现在最需要确认的是：你的主程序里到底创建的是：

```
C++
```

```
CNNArchitecture::LeNet
```

还是：

```
C++
```

```
CNNArchitecture::Enhanced
```

从你这份 `CNN.cpp` 本身看，如果走 Enhanced，才是现在更像正式训练用的结构；LeNet 更像保留的轻量教学/基线结构。

dropout的丢失是怎么实现的？是把随机一半的参数置零还是变成随机数

已思考几秒

Dropout 不是把参数置零，也不是把参数变成随机数。

它做的是：

在训练时，把某一层的输出值 / 激活值随机置零。

也就是说，丢掉的是这一轮 forward 里的某些“神经元输出”，不是权重参数本身。

比如 FC1 输出了 6 个数字：

```
[2.0, 0.5, 1.2, 3.0, 0.8, 4.0]
```

如果 `Dropout(0.5)`，表示大约随机丢掉一半。

可能生成一个随机 mask：

```
[1, 0, 1, 0, 1, 0]
```

然后输出变成：

```
[2.0, 0, 1.2, 0, 0.8, 0]
```

但你的代码用的是 **inverted dropout**，所以保留下来的值还会放大：

```
scale = 1 / (1 - rate)
```

如果 `rate = 0.5`：

```
scale = 1 / 0.5 = 2
```

所以实际输出是：

```
[4.0, 0, 2.4, 0, 1.6, 0]
```

为什么要乘以 2？

因为训练时随机丢了一半神经元，如果不放大，整体输出平均值会变小。

用 inverted dropout 后：

训练时：

随机丢掉一部分输出，保留下来的放大

推理时：

不丢弃，直接原样输出

这样推理阶段就不需要额外缩放。

回到你的代码，Enhanced 网络里是：

```
C++
```

```
FCLayer(128 * 4 * 4, 512)  
ReLU  
DropoutLayer(0.5)  
  
FCLayer(512, 256)  
ReLU  
DropoutLayer(0.5)  
  
FCLayer(256, classCount)  
Softmax
```

也就是说，Dropout 发生在：

```
FC1 → ReLU 之后  
FC2 → ReLU 之后
```

它随机置零的是：

```
ReLU 输出后的激活值
```

不是：

```
FC1 的权重  
FC2 的权重  
卷积核参数  
bias
```

这些参数仍然保留，只是这一次训练时，部分神经元输出暂时不参与后续计算。

反向传播时也一样。

如果某个位置 forward 时被 Dropout 丢掉了：

```
输出 = 0
```

那么 backward 时这个位置的梯度也会被置零：

```
gradient = 0
```

也就是：

```
这一次训练中，这个被丢掉的神经元不更新相关贡献
```

但下一张图片、下一个 batch，它可能又被保留下来。

所以 Dropout 的核心是：

训练时：

随机屏蔽一部分激活值

推理时：

不屏蔽，全部使用

参数：

不会被随机置零

不会变成随机数

一句话：

Dropout 是临时随机关闭一部分神经元输出，而不是破坏或随机化模型参数。